

1 Communication Asynchrone avec RabbitMQ

1.1 Pourquoi la communication asynchrone ?

Dans une architecture microservices, les services doivent communiquer entre eux. Deux approches existent :

Critere	Synchrone (REST)	Asynchrone (RabbitMQ)
Attente	Oui — le service attend	Non — continue sans attendre
Couplage	Fort	Faible
Disponibilite	Si service B tombe, A bloque	Service B peut etre hors ligne
Usage ideal	Lecture de donnees	Email, notifications, logs

Analogie : la boîte aux lettres

RabbitMQ fonctionne comme une **boîte aux lettres** :

- Le **Publisher** (expéditeur) depose un message dans la boîte
- Le **Consumer** (destinataire) recupere le message quand il est disponible
- Les deux n'ont pas besoin d'être actifs **en meme temps**

1.2 Principes de base de RabbitMQ

RabbitMQ

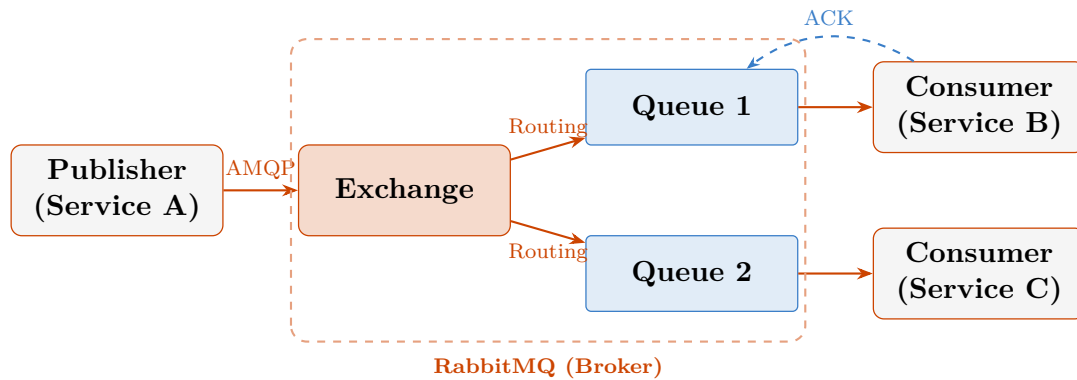
RabbitMQ est un **broker de messages** open source implementant le protocole **AMQP** (Advanced Message Queuing Protocol).

Son role : recevoir des messages de producteurs (*publishers*), les stocker dans des files d'attente (*queues*), et les distribuer aux consommateurs (*consumers*).

1.2.1 Les concepts fondamentaux

Concept	Description
Producer / Publisher	Service qui envoie les messages
Consumer	Service qui recoit et traite les messages
Queue	File d'attente ou les messages sont stockes
Exchange	Recoit les messages et les route vers les queues
Binding	Regle qui relie un exchange a une queue
Message	Donnees envoyees (texte, JSON, etc.)
Acknowledgement (ACK)	Confirmation de traitement d'un message

1.2.2 Flux d'un message dans RabbitMQ



1.2.3 Types d'Exchange

Type	Comportement	Usage
Direct	Route vers la queue dont la cle correspond exactement	Routage precis
Fanout	Route vers <i>toutes</i> les queues liees	Broadcast, notifications
Topic	Route selon un pattern (ex: <code>order.*</code>)	Filtrage flexible
Default	Route directement par nom de queue	Usage simple sans exchange explicite

1.3 Installation de RabbitMQ

1.3.1 Option 1 : Installation locale (Ubuntu/Debian)

```
# Installer RabbitMQ
sudo apt-get install -y rabbitmq-server

# Demarrer le service
sudo systemctl start rabbitmq-server
sudo systemctl enable rabbitmq-server    # Demarrage automatique

# Verifier le statut
sudo systemctl status rabbitmq-server

# Activer l'interface de gestion web (optionnel)
sudo rabbitmq-plugins enable rabbitmq_management
# Interface accessible sur : http://localhost:15672
# Identifiants par default : guest / guest
```

Listing 1: Installation de RabbitMQ sur Ubuntu

1.3.2 Option 2 : Installation avec Docker (Recommandee)

```
# Lancer RabbitMQ avec l'interface de gestion
docker run -d \
  --name rabbitmq \
  -p 5672:5672 \
  -p 15672:15672 \
  rabbitmq:3-management

# Port 5672 : port AMQP (connexion des applications)
# Port 15672 : interface web de gestion

# Verifier que le conteneur tourne
docker ps

# Voir les logs
docker logs rabbitmq
```

Listing 2: Lancer RabbitMQ avec Docker

✓ Interface de gestion RabbitMQ

Une fois lancee, accédez à **http://localhost:15672** avec :

- Utilisateur : guest
- Mot de passe : guest

Cette interface permet de surveiller les queues, messages et consumers en temps reel.

```
# Verifier l'etat de RabbitMQ
sudo rabbitmqctl status
```

```
# Lister les queues existantes
sudo rabbitmqctl list_queues

# Voir les utilisateurs
sudo rabbitmqctl list_users
```

Listing 3: Verification de l'installation

1.4 Installation du module client amqplib

amqplib

amqplib est le client Node.js le plus utilise pour interagir avec RabbitMQ via le protocole AMQP. Il fournit une API simple pour publier et consommer des messages.

```
# Creer et initialiser le projet
mkdir mon-service && cd mon-service
npm init -y

# Installer amqplib
npm install amqplib

# Installer dotenv pour les variables d'environnement
npm install dotenv
```

Listing 4: Initialisation du projet et installation

```
1 mon-service/
2 |-- publisher.js    # Service qui envoie des messages
3 |-- consumer.js    # Service qui recoit les messages
4 |-- .env           # Variables d'environnement
5 |-- package.json
```

Listing 5: Structure de projet recommandee

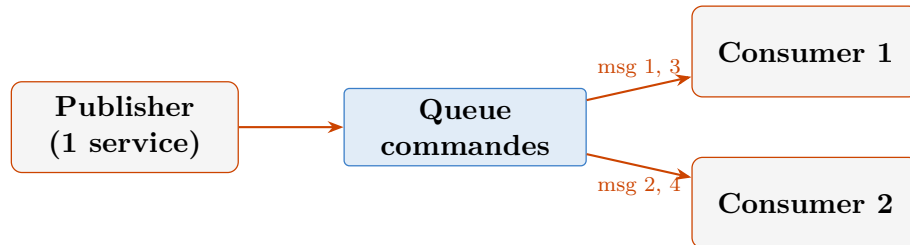
```
1 RABBITMQ_URL=amqp://localhost
2 QUEUE_NAME=commandes
```

Listing 6: Contenu du fichier .env

1.5 Mise en oeuvre de la communication

1.5.1 Pattern 1 : Work Queue (File de travail)

Ce pattern distribue les messages entre plusieurs consumers. Chaque message est traite par **un seul** consumer (round-robin).



Distribution automatique (Round-Robin)

```
1 const amqp = require('amqplib');
2
3 const RABBITMQ_URL = process.env.RABBITMQ_URL || 'amqp://
  localhost';
4 const QUEUE = 'commandes';
5
6 async function sendMessage(message) {
7   let connection;
8   try {
9     // 1. Etablir la connexion au serveur RabbitMQ
10    connection = await amqp.connect(RABBITMQ_URL);
11
12    // 2. Creer un channel (canal de communication)
13    const channel = await connection.createChannel();
14
15    // 3. Declarer la queue (la creer si elle n'existe pas)
16    // durable: true = la queue survit au redemarrage de
  RabbitMQ
17    await channel.assertQueue(QUEUE, { durable: true });
18
19    // 4. Convertir le message en Buffer et l'envoyer
20    const msgBuffer = Buffer.from(JSON.stringify(message));
21    channel.sendToQueue(QUEUE, msgBuffer, {
22      persistent: true // Message sauvegarde sur disque
23    });
24
25    console.log('[Publisher] Message envoye:', message);
26
27    // 5. Fermer le channel et la connexion
28    await channel.close();
29    await connection.close();
30
31  } catch (error) {
32    console.error('[Publisher] Erreur:', error.message);
33    if (connection) await connection.close();
34  }
35 }
```

```
36
37 // Exemple d'utilisation
38 sendMessage({
39   type: 'NOUVELLE_COMMANDE',
40   id: 42,
41   produit: 'Laptop',
42   quantite: 1
43 });
```

Listing 7: publisher.js — Envoi de messages a la queue

```
1  const amqp = require('amqplib');
2
3  const RABBITMQ_URL = process.env.RABBITMQ_URL || 'amqp://
   localhost';
4  const QUEUE = 'commandes';
5
6  async function startConsumer() {
7    try {
8      // 1. Connexion et channel
9      const connection = await amqp.connect(RABBITMQ_URL);
10     const channel = await connection.createChannel();
11
12     // 2. Declarer la meme queue que le publisher
13     await channel.assertQueue(QUEUE, { durable: true });
14
15     // 3. Traiter un seul message a la fois (evite la
   surcharge)
16     channel.prefetch(1);
17
18     console.log('[Consumer] En attente de messages sur "${
   QUEUE}"...');
19
20     // 4. Ecouter les messages en continu
21     channel.consume(QUEUE, async (msg) => {
22       if (msg === null) return; // Consumer annule
23
24       // 5. Decoder le message JSON
25       const content = JSON.parse(msg.content.toString());
26       console.log('[Consumer] Message reçu:', content);
27
28       try {
29         // 6. Traiter le message (logique metier)
30         await traiterCommande(content);
31
32         // 7. ACK : confirmer le traitement reussi
33         channel.ack(msg);
34         console.log('[Consumer] Message traite avec
   succes');
35
36       } catch (error) {
37         // 8. NACK : rejeter et remettre dans la queue
38         channel.nack(msg, false, true);
39         console.error('[Consumer] Erreur:', error.message);
```

```
);  
40     }  
41     });  
42  
43     } catch (error) {  
44         console.error('[Consumer] Connexion impossible:', error.  
message);  
45         // Reconnexion automatique apres 5 secondes  
46         setTimeout(startConsumer, 5000);  
47     }  
48 }  
49  
50 async function traiterCommande(commande) {  
51     // Logique metier (ex: enregistrement en BDD)  
52     console.log('Traitement commande n${commande.id} : ${commande  
.produit}');  
53     await new Promise(resolve => setTimeout(resolve, 500));  
54 }  
55  
56 startConsumer();
```

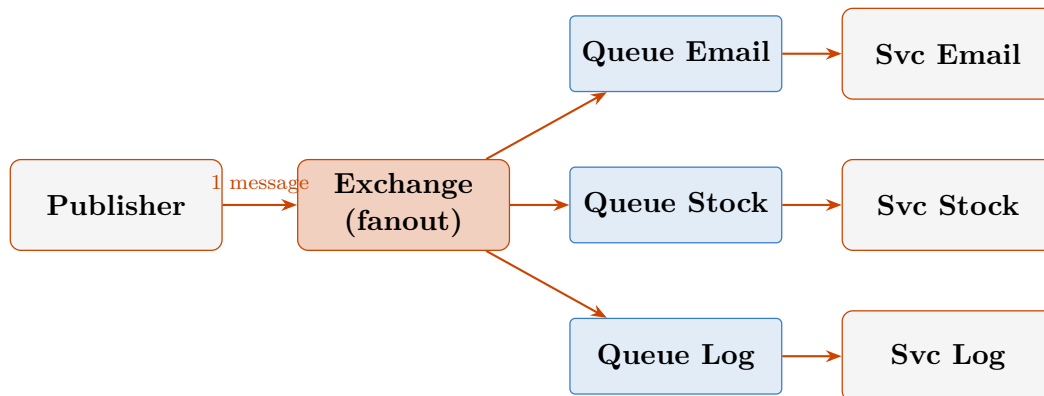
Listing 8: consumer.js — Reception et traitement des messages

```
# Terminal 1 : Lancer le consumer (reste en ecoute)  
node consumer.js  
# [Consumer] En attente de messages sur "commandes"...  
  
# Terminal 2 : Envoyer un message  
node publisher.js  
# [Publisher] Message envoye: { type: 'NOUVELLE_COMMANDE', id:  
42, ... }
```

Listing 9: Executer le publisher et le consumer

1.5.2 Pattern 2 : Publish / Subscribe (Fanout)

Ce pattern envoie le même message à **tous** les consumers abonnés. Utile pour les notifications ou les événements métier.



Copie du message vers toutes les queues simultanément

```

1  const amqp = require('amqplib');
2
3  async function broadcast(eventement) {
4      const connection = await amqp.connect('amqp://localhost');
5      const channel    = await connection.createChannel();
6
7      const EXCHANGE = 'evenements_boutique';
8
9      // Declarer un exchange de type 'fanout'
10     await channel.assertExchange(EXCHANGE, 'fanout', { durable:
false });
11
12     // Envoyer le message (routing key vide avec fanout)
13     channel.publish(EXCHANGE, '', Buffer.from(JSON.stringify(
eventement)));
14
15     console.log('[Publisher] Evenement diffuse:', eventement);
16     await channel.close();
17     await connection.close();
18 }
19
20 broadcast({ type: 'COMMANDE_VALIDEE', commandeId: 99, montant:
1200 });
  
```

Listing 10: publisher_fanout.js — Diffusion vers tous les services

```

1  const amqp = require('amqplib');
2
3  async function subscribe(nomService, handler) {
4      const connection = await amqp.connect('amqp://localhost');
5      const channel    = await connection.createChannel();
6
7      const EXCHANGE = 'evenements_boutique';
8      await channel.assertExchange(EXCHANGE, 'fanout', { durable:
false });
9
  
```



```
10 // Creer une queue temporaire unique pour ce consumer
11 const { queue } = await channel.assertQueue('', { exclusive:
true });
12
13 // Relier la queue a l'exchange (binding)
14 await channel.bindQueue(queue, EXCHANGE, '');
15
16 console.log(`[${nomService}] En attente d'evenements...`);
17
18 channel.consume(queue, (msg) => {
19     const data = JSON.parse(msg.content.toString());
20     handler(data);
21 }, { noAck: true });
22 }
23
24 // Chaque service cree son propre consumer
25 subscribe('ServiceEmail', (event) => {
26     if (event.type === 'COMMANDE_VALIDEE') {
27         console.log(`[Email] Confirmation commande n${event.
commandeId}`);
28     }
29 });
30
31 subscribe('ServiceStock', (event) => {
32     if (event.type === 'COMMANDE_VALIDEE') {
33         console.log(`[Stock] Mise a jour apres commande n${event.
commandeId}`);
34     }
35 });
```

Listing 11: consumer_fanout.js — Abonnement aux evenements

1.5.3 Connexion robuste avec reconnexion automatique

```
1 const amqp = require('amqplib');
2
3 class RabbitMQService {
4   constructor(url, queueName) {
5     this.url = url;
6     this.queueName = queueName;
7     this.connection = null;
8     this.channel = null;
9   }
10
11   async connect() {
12     try {
13       this.connection = await amqp.connect(this.url);
14       this.channel = await this.connection.createChannel
15       ();
16       await this.channel.assertQueue(this.queueName, {
17         durable: true });
18
19       // Reconnexion automatique si la connexion se ferme
20       this.connection.on('close', () => {
21         console.warn('[RabbitMQ] Connexion fermee.
22         Reconnexion...');
23         setTimeout(() => this.connect(), 5000);
24       });
25
26       this.connection.on('error', (err) => {
27         console.error('[RabbitMQ] Erreur:', err.message);
28       });
29
30       console.log('[RabbitMQ] Connecte avec succes');
31       return this;
32     } catch (error) {
33       console.error('[RabbitMQ] Connexion echouee:', error.
34       message);
35       setTimeout(() => this.connect(), 5000);
36     }
37   }
38
39   async publish(message) {
40     if (!this.channel) throw new Error('Channel non
41     initialise');
42     return this.channel.sendToQueue(
43       this.queueName,
44       Buffer.from(JSON.stringify(message)),
45       { persistent: true }
46     );
47   }
48
49   async consume(handler) {
```

```
46     if (!this.channel) throw new Error('Channel non
initialise');
47     this.channel.prefetch(1);
48     this.channel.consume(this.queueName, async (msg) => {
49         if (!msg) return;
50         const data = JSON.parse(msg.content.toString());
51         try {
52             await handler(data);
53             this.channel.ack(msg);
54         } catch (err) {
55             this.channel.nack(msg, false, true);
56         }
57     });
58 }
59 }
60
61 // Utilisation
62 async function main() {
63     const rabbit = new RabbitMQService('amqp://localhost', '
commandes');
64     await rabbit.connect();
65     await rabbit.publish({ id: 1, produit: 'Clavier' });
66 }
67
68 main();
```

Listing 12: Service RabbitMQ avec gestion des erreurs

1.6 Synthèse RabbitMQ

Points cles a retenir

1. **Publisher** : envoie un message et continue sans attendre
2. **Consumer** : recoit et traite le message independamment
3. **Queue** : stocke les messages jusqu'a leur traitement
4. **ACK** : confirme qu'un message a bien ete traite
5. **NACK** : rejette un message et le remet dans la queue
6. **prefetch(1)** : un seul message a la fois par consumer

Action	Methode amqplib
Se connecter	<code>amqp.connect(url)</code>
Creer un canal	<code>connection.createChannel()</code>
Declarer une queue	<code>channel.assertQueue(nom, options)</code>
Envoyer un message	<code>channel.sendToQueue(queue, Buffer)</code>
Ecouter les messages	<code>channel.consume(queue, callback)</code>
Confirmer (ACK)	<code>channel.ack(msg)</code>
Rejeter (NACK)	<code>channel.nack(msg, false, true)</code>